

SIMATS ENGINEERING

ASSESSMENT TOOL 3(LOW)-Application Based Question

COURSE NAME: R PROGRAMMING

COURSE CODE: ITA04

COURSE OUTCOME COVERED

CO: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively.

(Bloom Level -BL2- BL4)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks: 100

1. Student Performance Analysis

A college stores the marks of 10 students in Mathematics, Physics, and Chemistry. Write an R program to:

- Store the marks in a matrix.
- Calculate subject-wise averages.
- Find the highest and lowest scorer in each subject.
- Display students scoring above the overall average.

Solution:

```
1 marks <- matrix(c(78,82,91,65,88,74,85,79,92,81,80,76,89,68,91,72,84,77,90,83,75,69,88,71,86,73,82,80,87,79), ncol=3)
2 colnames(marks) <- c("Maths","Physics","Chemistry")
3 avg <- colMeans(marks)
4 print(avg)
5 print(apply(marks,2,max))
6 print(apply(marks,2,min))
7 overall <- mean(marks)
8 print(marks[rowMeans(marks) > overall, ])
```

Output:

```
> marks <- matrix(c(78,82,91,65,88,74,85,79,92,81,80,76,89,68,91,72,84,77,90,83,75,69,88,71,86,73,82,80,87,79), ncol=3)
> colnames(marks) <- c("Maths","Physics","Chemistry")
> avg <- colMeans(marks)
> print(avg)
      Maths  Physics Chemistry
      81.5    81.0    79.0
> print(apply(marks,2,max))
      Maths  Physics Chemistry
      92     91     88
> print(apply(marks,2,min))
      Maths  Physics Chemistry
      65     68     69
> overall <- mean(marks)
> print(marks[rowMeans(marks) > overall, ])
      Maths Physics Chemistry
[1,]    91     89     88
[2,]    88     91     86
[3,]    85     84     82
[4,]    92     90     87
[5,]    81     83     79
```

2. Employee Salary Processing

An organization maintains employee details including Employee ID, Name, Basic Pay, and Allowance.

Write an R program using appropriate data structures to:

- Calculate Gross Salary.
- Identify employees earning above ₹50,000.
- Display the employee with the highest salary.
- Generate a summary report.

Solution:

```
10 emp <- data.frame(ID=1:5, Name=c("Yashwitha","Rahul","Priya","Arun","Sneha"), Basic=c(35000,45000,30000,50000,40000), Allowance=c(5000,7000,4000,8000,6000))
11 emp$Gross <- emp$Basic + emp$Allowance
12 print(emp)
13 print(emp[emp$Gross > 50000, ])
14 print(emp[which.max(emp$Gross), ])
15 summary(emp)
```

Output:

```
> emp <- data.frame(ID=1:5, Name=c("Yashwitha","Rahul","Priya","Arun","Sneha"), Basic=c(35000,45000,30000,50000,40000), Allowance=c(5000,7000,4000,8000,6000))
> emp$Gross <- emp$Basic + emp$Allowance
> print(emp)
  ID Name Basic Allowance Gross
1  1 Yashwitha 35000    5000 40000
2  2 Rahul 45000    7000 52000
3  3 Priya 30000    4000 34000
4  4 Arun 50000    8000 58000
5  5 Sneha 40000    6000 46000
> print(emp[emp$Gross > 50000, ])
  ID Name Basic Allowance Gross
2  2 Rahul 45000    7000 52000
4  4 Arun 50000    8000 58000
> print(emp[which.max(emp$Gross), ])
  ID Name Basic Allowance Gross
4  4 Arun 50000    8000 58000
> summary(emp)
      ID      Name      Basic      Allowance      Gross
Min.   :1  Length:5      Min.   :30000    Min.   :4000    Min.   :34000
1st Qu.:2   Class :character 1st Qu.:35000    1st Qu.:5000    1st Qu.:40000
Median :3   Mode  :character Median :40000    Median :6000    Median :46000
Mean    :3              Mean  :40000    Mean  :6000    Mean  :46000
3rd Qu.:4              3rd Qu.:45000    3rd Qu.:7000    3rd Qu.:52000
Max.    :5              Max.   :50000    Max.   :8000    Max.   :58000
```

3. Weather Data Analysis Using Arrays

Daily temperature data for three cities is collected for 7 days.

Write an R program to:

- Store the data in a 3-dimensional array.
- Find the average temperature for each city.
- Identify the hottest day.
- Compare city-wise temperature variations.

Solution:

```
17 temp <- array(c(30,32,31,29,33,34,30,28,29,30,31,32,33,34,27,28,29,30,31,32,33), dim=c(7,3,1))
18 dimnames(temp) <- list(Day=1:7, City=c("Chennai","Delhi","Mumbai"), Data="Temp")
19 print(apply(temp, 2, mean))
20 print(which(temp == max(temp), arr.ind=TRUE))
21 print(apply(temp, 2, sd))
22 print(temp)
```

Output:

```
> temp <- array(c(30,32,31,29,33,34,30,28,29,30,31,32,33,34,27,28,29,30,31,32,33), dim=c(7,3,1))
> dimnames(temp) <- list(Day=1:7, City=c("Chennai","Delhi","Mumbai"), Data="Temp")
> print(apply(temp, 2, mean))
Chennai Delhi Mumbai
31.28571 31.00000 30.00000
> print(which(temp == max(temp), arr.ind=TRUE))
Day City Data
6 6 1 1
7 7 2 1
> print(apply(temp, 2, sd))
Chennai Delhi Mumbai
1.799471 2.160247 2.160247
> print(temp)
, , Data = Temp

City
Day Chennai Delhi Mumbai
1 30 28 27
2 32 29 28
3 31 30 29
4 29 31 30
5 33 32 31
6 34 33 32
7 30 34 33
```

4. Product Inventory Management

A store maintains product information such as Product ID, Product Name, Quantity, and Price.

Using lists and vectors in R:

- Calculate inventory value for each product.
- Find products with stock below a threshold value.

- Generate a stock summary report.

Solution:

```
24 products <- list(ID=1:5, Name=c("Pen","Book","Bag","Bottle","Pencil"), Quantity=c(50,30,20,40,100), Price=c(10,50,500,100,5))
25 value <- products$Quantity * products$Price
26 print(data.frame(Product=products$Name, InventoryValue=value))
27 print(data.frame(Product=products$Name, Quantity=products$Quantity)[products$Quantity < 30, ])
28 summary(data.frame(Quantity=products$Quantity, Price=products$Price, Value=value))
```

Output:

```
> products <- list(ID=1:5, Name=c("Pen","Book","Bag","Bottle","Pencil"), Quantity=c(50,30,20,40,100), Price=c(10,50,500,100,5))
> value <- products$Quantity * products$Price
> print(data.frame(Product=products$Name, InventoryValue=value))
  Product InventoryValue
1    Pen             500
2   Book            1500
3    Bag           10000
4 Bottle            4000
5 Pencil             500
> print(data.frame(Product=products$Name, Quantity=products$Quantity)[products$Quantity < 30, ])
  Product Quantity
3    Bag         20
> summary(data.frame(Quantity=products$Quantity, Price=products$Price, Value=value))
      Quantity      Price      Value
Min.   : 20   Min.    : 5   Min.    : 500
1st Qu.: 30   1st Qu.: 10   1st Qu.: 500
Median : 40   Median : 50   Median : 1500
Mean   : 48   Mean   :133   Mean   : 3300
3rd Qu.: 50   3rd Qu.:100   3rd Qu.: 4000
Max.   :100   Max.    :500   Max.    :10000
```

5. Bank Account Processing Using Functions

Develop an R program with user-defined functions to:

- Deposit money.
 - Withdraw money.
 - Check account balance.
 - Display transaction history.
- Use suitable variables and functions.

Solution:

```
30 balance <- 10000; history <- c()
31 deposit <- function(x){balance <- balance + x; history <- c(history,paste("Deposit",x))}
32 withdraw <- function(x){if(x<=balance){balance <- balance - x; history <- c(history,paste("Withdraw",x))}}
33 checkBalance <- function(){cat("Balance =",balance,"\n")}
34 transactions <- function(){print(history)}
35 deposit(5000)
36 withdraw(2000)
37 checkBalance()
38 transactions()
```

Output:

```
> balance <- 10000; history <- c()
> deposit <- function(x){balance <- balance + x; history <- c(history,paste("Deposit",x))}
> withdraw <- function(x){if(x<=balance){balance <- balance - x; history <- c(history,paste("Withdraw",x))}}
> checkBalance <- function(){cat("Balance =",balance,"\n")}
> transactions <- function(){print(history)}
> deposit(5000)
> withdraw(2000)
> checkBalance()
Balance = 13000
> transactions()
[1] "Deposit 5000" "Withdraw 2000"
```

6. Hospital Patient Data Analysis

A hospital stores patient details including Age, Gender, and Treatment Cost.
Write an R program to:

- Store the information in a data frame.
- Calculate average treatment cost.
- Find patients above 60 years of age.
- Generate statistical summaries.

Solution:

```
40 name <- c("Yashwitha","Rahul","Priya","Arun","Sneha")
41 age <- c(65,45,72,58,68)
42 gender <- c("F","M","F","M","F")
43 cost <- c(50000,30000,45000,25000,60000)
44 patient <- data.frame(name, age, gender, cost)
45 print(patient)
46 cat("Average Cost =", mean(cost), "\n")
47 print(patient[patient$age > 60, ])
48 summary(patient)
```

Output:

```
> name <- c("Yashwitha","Rahul","Priya","Arun","Sneha")
> age <- c(65,45,72,58,68)
> gender <- c("F","M","F","M","F")
> cost <- c(50000,30000,45000,25000,60000)
> patient <- data.frame(name, age, gender, cost)
> print(patient)
  name age gender cost
1 Yashwitha 65      F 50000
2   Rahul  45      M 30000
3   Priya  72      F 45000
4   Arun  58      M 25000
5   Sneha  68      F 60000
> cat("Average Cost =", mean(cost), "\n")
Average Cost = 42000
> print(patient[patient$age > 60, ])
  name age gender cost
1 Yashwitha 65      F 50000
3   Priya  72      F 45000
5   Sneha  68      F 60000
> summary(patient)
      name      age      gender      cost
Length:5      Min.   :45.0  Length:5      Min.   :25000
Class :character 1st Qu.:58.0  Class :character 1st Qu.:30000
Mode  :character Median :65.0  Mode  :character Median :45000
                        Mean  :61.6                        Mean  :42000
                        3rd Qu.:68.0                        3rd Qu.:50000
                        Max.  :72.0                        Max.   :60000
```

7. Logical Vector-Based Voting Eligibility System

Create a dataset containing Name and Age of citizens.

Using logical vectors:

- Determine voting eligibility ($\text{Age} \geq 18$).
- Count eligible and ineligible voters.
- Display the percentage of eligible voters.

Solution:

```
50 name <- c("Yashwitha","Rahul","Priya","Arun","Sneha")
51 age <- c(20,16,25,17,30)
52 eligible <- age >= 18
53 voters <- data.frame(name, age, eligible)
54 print(voters)
55 cat("Eligible Voters =", sum(eligible), "\n")
56 cat("Ineligible Voters =", sum(!eligible), "\n")
57 cat("Eligible Percentage =", (sum(eligible)/length(age))*100, "%\n")
```

Output:

```
> name <- c("Yashwitha","Rahul","Priya","Arun","Sneha")
> age <- c(20,16,25,17,30)
> eligible <- age >= 18
> voters <- data.frame(name, age, eligible)
> print(voters)
  name age eligible
1 Yashwitha 20     TRUE
2   Rahul  16    FALSE
3   Priya  25     TRUE
4   Arun  17    FALSE
5   Sneha  30     TRUE
> cat("Eligible Voters =", sum(eligible), "\n")
Eligible Voters = 3
> cat("Ineligible Voters =", sum(!eligible), "\n")
Ineligible Voters = 2
> cat("Eligible Percentage =", (sum(eligible)/length(age))*100, "%\n")
Eligible Percentage = 60 %
```

8. Matrix-Based Sales Analysis

A company records monthly sales for four products across three regions.

Write an R program to:

- Store the sales data in a matrix.
- Calculate region-wise and product-wise sales totals.
- Identify the best-performing product.
- Display the region with maximum sales.

Solution:

```
59 sales <- matrix(c(120,150,180,200,130,160,190,210,140,170,200,220), nrow=3, byrow=TRUE)
60 colnames(sales) <- c("Product1","Product2","Product3","Product4")
61 rownames(sales) <- c("North","South","East")
62 print(sales)
63 cat("Region-wise Sales =", rowSums(sales), "\n")
64 cat("Product-wise Sales =", colSums(sales), "\n")
65 cat("Best Product =", names(which.max(colSums(sales))), "\n")
66 cat("Best Region =", names(which.max(rowSums(sales))), "\n")
```

Output:

```
> sales <- matrix(c(120,150,180,200,130,160,190,210,140,170,200,220), nrow=3, byrow=TRUE)
> colnames(sales) <- c("Product1","Product2","Product3","Product4")
> rownames(sales) <- c("North","South","East")
> print(sales)
      Product1 Product2 Product3 Product4
North      120      150      180      200
South      130      160      190      210
East       140      170      200      220
> cat("Region-wise Sales =", rowSums(sales), "\n")
Region-wise Sales = 650 690 730
> cat("Product-wise Sales =", colSums(sales), "\n")
Product-wise Sales = 390 480 570 630
> cat("Best Product =", names(which.max(colSums(sales))), "\n")
Best Product = Product4
> cat("Best Region =", names(which.max(rowSums(sales))), "\n")
Best Region = East
```

9. Library Management Using S3 Class

Design an S3 class named **Book** with attributes:

- Book ID
- Title
- Author
- Number of Copies

Write methods to:

- Display book details.
- Update available copies.
- Search for a specific book.

Solution:

```
68 Book <- function(id,title,author,copies) structure(list(ID=id,Title=title,Author=author,Copies=copies), class="Book")
69 print.Book <- function(x) print(unlist(x))
70 update <- function(x,n){x$Copies <- x$Copies+n; print(x)}
71 search <- function(x,t){if(x$Title==t) print(x) else print("Book Not Found")}
72 b1 <- Book(101,"R Programming","Yashwitha",10)
73 print(b1)
74 update(b1,5)
75 search(b1,"R Programming")
```

Output:

```
> Book <- function(id,title,author,copies) structure(list(ID=id,Title=title,Author=author,Copies=copies), class="Book")
> print.Book <- function(x) print(unlist(x))
> update <- function(x,n){x$Copies <- x$Copies+n; print(x)}
> search <- function(x,t){if(x$Title==t) print(x) else print("Book Not Found")}
> b1 <- Book(101,"R Programming","Yashwitha",10)
> print(b1)
      ID      Title      Author      Copies
"101" "R Programming" "Yashwitha"      "10"
> update(b1,5)
      ID      Title      Author      Copies
"101" "R Programming" "Yashwitha"      "15"
> search(b1,"R Programming")
      ID      Title      Author      Copies
"101" "R Programming" "Yashwitha"      "10"
```

10. University Information System Using S4 Class and Reference Class

Create an S4 class named **Student** with slots:

- Roll Number
- Name
- CGPA

Create a Reference Class to:

- Add student records.
- Modify CGPA.
- Display student details.
- Find students with CGPA greater than 8.0.

Solution:

```
77 setClass("Student", slots=list(RollNo="numeric", Name="character", CGPA="numeric"))
78 StudentRef <- setRefClass("StudentRef", fields=list(records="list"))
79 obj <- StudentRef$new(records=list(new("Student", RollNo=101, Name="Yashwitha", CGPA=9.2)))
80 obj$records[[1]]@CGPA <- 9.5
81 show(obj$records[[1]])
82 if(obj$records[[1]]@CGPA > 8.0)
83   cat("Student with CGPA > 8.0 :", obj$records[[1]]@Name, "\n")
```

Output:

```
> setClass("Student", slots=list(RollNo="numeric", Name="character", CGPA="numeric"))
> StudentRef <- setRefClass("StudentRef", fields=list(records="list"))
> obj <- StudentRef$new(records=list(new("Student", RollNo=101, Name="Yashwitha", CGPA=9.2)))
> obj$records[[1]]@CGPA <- 9.5
> show(obj$records[[1]])
An object of class "Student"
Slot "RollNo":
[1] 101

Slot "Name":
[1] "Yashwitha"

Slot "CGPA":
[1] 9.5

> if(obj$records[[1]]@CGPA > 8.0)
+   cat("Student with CGPA > 8.0 :", obj$records[[1]]@Name, "\n")
Student with CGPA > 8.0 : Yashwitha
```

Code of Conduct:

I, Savitha R / 192421368 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Savitha R

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand